



GRUPPENNUMMER: 308

Neuronales Netz für den kalifornischen Hauspreis-Datensatz

Übrige Gruppenmitglieder:

Florian Adamczyk

Björn Becker

Felix Hollfoth

Kolja Scriba

Protokoll von:

Felix Neumann (Sprecherin)

felix.neumann@stud.uni-giessen.de

Matrikel Nr.: 16639

Modul: Künstliche Intelligenz I
Dozent: Prof. Dr. Christian Heiliger
Semester: Wintersemester 2025/26

INHALTSVERZEICHNIS

1	Zusammenfassung des Logbuchs	3
1.1	Fragestellungen	3
1.2	Ansätze und Methoden <i>bayesianische Optimierung • Feature Engineerings</i>	3
1.3	Ergebnisse <i>Vergleich mit linearer Regression</i>	3
1.4	Schwierigkeiten und Grenzen <i>Technische Herausforderungen • Methodische Grenzen</i>	5
1.5	Zwischenstand und Ausblick	5
2	Einschätzung der Gruppenmitglieder	6
3	Zusammenfassung der Ergebnisse der Gruppe	7
3.1	Explorative Datenanalyse, -Bereinigung und -Darstellung (Florian) <i>Analyse • Bereinigung • Skalierung • Darstellungen bereinigter Daten</i>	7
3.2	Lineare Regression	8
3.3	LASSO und Ridge (Florian, Kolja)	8
3.4	Decision Tree Regression (Florian)	8
3.5	Ensemble (Florian)	8
3.6	K-Nearest Neighbors Regression (Florian)	8
3.7	Neuronale Netze <i>Architektur • Feature engineering (Felix N., Björn, Felix H.) • Gesamtvergleich mit der Linearen Regression</i>	8
4	Protokoll der Gruppentreffen	10
5	Logbuch	11
6	Quellen und Hilfsmittel	25

Neuronales Netz für den kalifornischen Hauspreis-Datensatz

Felix Neumann^a

^aJustus Liebig Universität, Matrikelnr. 116639

Gruppe 308 – KI I Projekt WS 2025/26

Zusammenfassung—Diese Zusammenfassung dokumentiert den individuellen Projektfortschritt basierend auf dem geführten Logbuch im Rahmen des KI I-Projekts zur Vorhersage kalifornischer Hauspreise mittels neuronaler Netze. Das vollständige Logbuch befindet sich im Anhang des Portfolios. Dabei wurden mehr als akzeptable Ergebnisse erzielt. Mein Anteil, Vorgehen, die Gedankengänge und Problematiken sind hier unvollständig (im Logbuch vollständig) dokumentiert. Das Bestergebnis meines Neuronalen Netzes auf die Testdaten, übertraf dabei mit einem R^2 -Score von 0.7958 das der linearen Regression von 0.6326 deutlich.

Keywords—Neuronales Netz, California Housing, TensorFlow, Regression, Machine Learning

1.1. Fragestellungen

Diese Zusammenfassung dokumentiert den individuellen Projektfortschritt basierend auf dem geführten Logbuch. Die zentrale Fragestellung der Arbeit lag in der Optimierung neuronaler Netze für den California-Housing-Datensatz und einem anschließenden Vergleich dieser zu der linearen Regression. Als Zielvariable diente dabei der *MedHouseVal*. Basierend auf dem Fortschritt der Teammitglieder, in den Bereichen der besten Hyperparameterwahl, sowie der generellen Architektur und insbesondere der Erkenntnis, dass eine isolierte Hyperparameterwahl nicht zielführend war entstanden verschiedene Ansätze für das Vorgehen. Neben einer Simultanoptimierung der Hyperparameter die effizienter und Ressourcensparender läuft, als die bereits erprobten Grid- und Randomsearchvarianten, sollte auch an den Features selbst Veränderungen ausprobiert werden. Dabei wurden folgende spezifische Teilfragen untersucht:

- Wie lässt sich die Hyperparameter-Suche durch bayesianische Optimierung im Vergleich zu Grid- oder Random-Search effizienter gestalten?
- Kann das Netz durch Feature engineering weiter verbessert werden? Welchen Einfluss haben neu generierte Features (wie *Bedrooms_per_Room* oder geografische Distanzen zu Los Angeles und San Francisco) auf die Modellperformance?
- Ab welcher *Gini Importance* können Features vernachlässigt werden, ohne dass das neuronale Netz an Vorhersagekraft verliert?

1.2. Ansätze und Methoden

1.2.1. bayesianische Optimierung

Zur grundlegenden Orientierung beim Aufbau und der Theorie der neuronalen Netze diente einschlägige Fachliteratur [4]. Die anderen Gruppenmitglieder (insbesondere Felix H.) versuchten zunächst durch isolierte Hyperparameterwahl sinnvolle Zusammensetzungen für die Architektur zu finden. Später erfolgte eine zielgerichtete Suche mit Grid- und Randomsearch. Da diese sehr zeitaufwändig nach leistungsfähigen Hyperparameterzusammensetzungen suchten, recherchierte ich weitere Möglichkeiten zur Beschleunigung des Vorganges. Als Resultat wurde die bayesianische Optimierung mittels KerasTuner [3] implementiert. Dieser Ansatz wählt Hyperparameter basierend auf vergangenen Trainingsläufen aus, was die Rechenzeit drastisch reduziert hat. Zunächst wurden einfache, flache sequentielle Modelle [6] trainiert, um das Risiko von Vanishing Gradients und Overfitting zu minimieren. Im weiteren Verlauf wurde die Breite der

Netzwerke auf bis zu 256 Neuronen erweitert und Regularisierungstechniken wie Dropout sowie die LeakyReLU-Aktivierungsfunktion getestet. Auch mit verschiedenen Tiefen wurden experimentiert. Zum Schluss erlangte das beste Ergebnis ein Netz mit drei Hidden Layern und einer batch-size von 32, wie sie erfolgreich durch die Kommotionen als geeignet ermittelt wurde. Versuche mit größeren Batch-sizes wurden zwar durchgeführt, gingen jedoch bei allen Versuchen zulasten des R^2 – Score. Zudem kam ein dynamischer Learning-Rate-Scheduler in Verbindung mit [early Stopping] und der Adam-Optimizer zum Einsatz. Anhand des Plot "Fig. 3" lässt sich sehr gut die Funktion des eingesetzten Earlystoppings während eines beispielhaften Modelldurchlaufs erkennen.

1.2.2. Feature Engineerings

Zunächst erfolgte eine allgemeine Featureanalyse. Ganz klar zu erkennen ist die Dominanz des Medium Income Features bei einem Vergleich der Featureimportances. Es drängte sich die Frage auf, ob aus den restlichen Features noch mehr Information zu gewinnen sei. Durch eine Division der Features *AveBedrms* und *AveRooms* errechnete ich das Feature *Bedrooms_per_Room*. Die Analyse der aktuellen Wohnraumpreise in Kalifornien zeigte eine klare Tendenz hoher Kosten in der Nähe von Los Angeles und San Francisco. Die Frage war nun, ob dies auch aus dem Datensatz mit Daten der 90er hervor geht. Aus den vorhandenen Features für Längen und Breitengrad konnte jeweils ein Feature für die relative Nähe zu den beiden Großstädten ermittelt werden. Zusätzlich wurden diese auch noch auf einfache Weise verrechnet und ein weiteres geografisches Feature *Lat_x_Lon* konnte hinzugefügt werden. Eine weitere Feststellung war die rechtsschiefe Verteilung der Population. Da neuronale Netze mit schiefen Verteilungen etwas Schwierigkeiten haben können, wurde Testweise eine logarithmische Betrachtung *log_Population*. Zur Evaluation dieser methodischen Anpassungen wurde ein Random-Forest-Regressor trainiert, um die *Feature Importance* Fig. 4 zu analysieren und irrelevante Merkmale zu identifizieren. Die Korrelation zur Zielvariablen kann durch eine Fig. 1 abgelesen werden. Darauf zu erkennen ist die Dominanz der Korrelation des Median Einkommens zur Zielvariable, sowie eine nichttriviale Korrelation der neu eingeführten Distanz Features *Dist_to_SF* und *Dist_to_LA*.

1.3. Ergebnisse

Der Plot "Fig. 2" dient als Beispielresiduenverteilung aller erstellter Modelle. Diese unterscheiden sich nur unwesentlich und haben alle eine sehr ähnliche Verteilung.

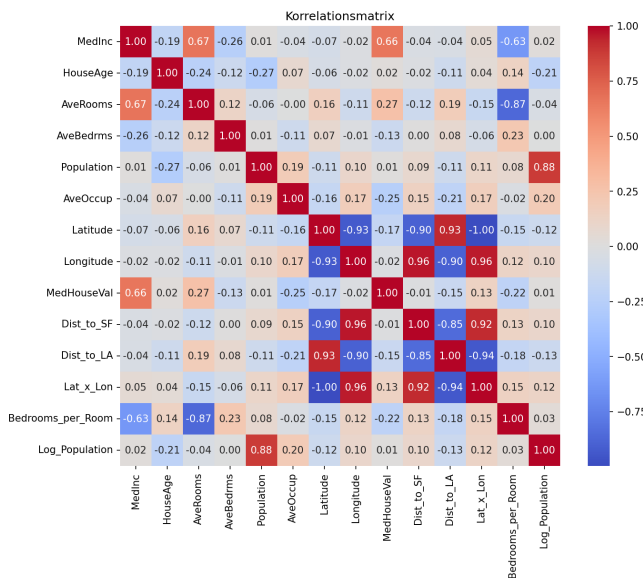


Abbildung 1. Korrelationskarte

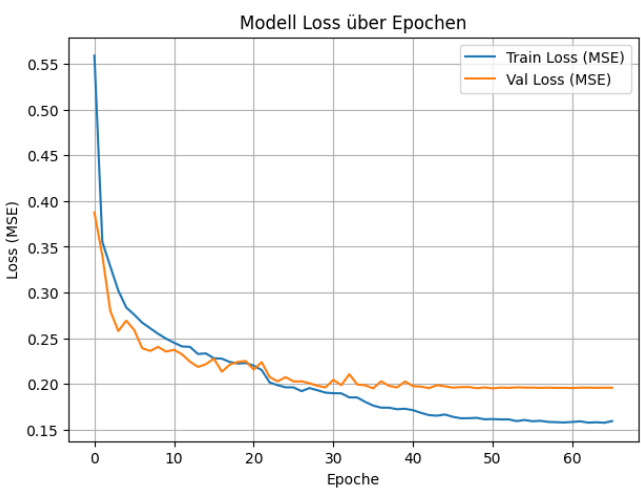


Abbildung 3. Loss über die Verschiedenen Epochen

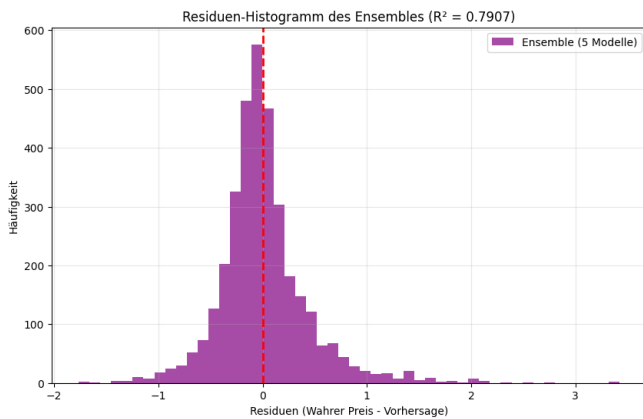


Abbildung 2. Beispiel eines Residuenhistogramms

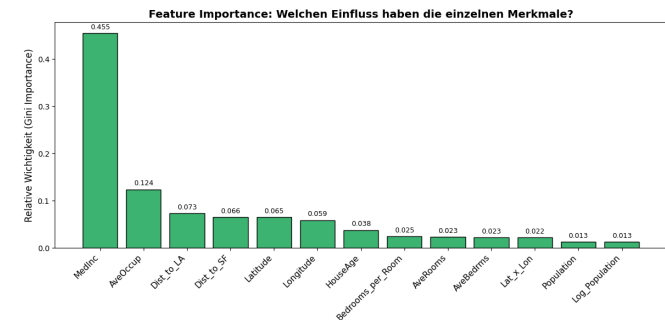


Abbildung 4. Feature Importance Analyse (inkl. Distanz-Features). Die Grafik verdeutlicht den verhältnismäßig geringen Einfluss der Populations-Features im Vergleich zum Medianeinkommen.

Das Residuendiagramm (Fig. 2) zeigt eine Zentrierung der Werte um die Nulllinie. Das Modell hat demnach keinen systematischen Bias und unterschätzt Werte ähnlich oft, wie es sie überschätzt. Obgleich der Großteil des Fehlers in einem engen Bereich liegt, zeigt er keine perfekte Normalverteilung, sondern hat "heavy tails". Eine solche leicht Rechtsschiefe Verteilung bedeutet, dass die Preise der Häuser im Zweifel leicht unterschätzt werden. Es sind auch einige Outlier (Residuen>2) zu erkennen deren Preis massiv unterschätzt wurde. Dies könnte an dem Fehlen von Features liegen, welche diese Preissteigungen erklären könnten.

Anhand des Loss-Plot (Fig. 3) lässt sich sehr gut die Funktion des eingesetzten Earlystoppings während eines beispielhaften Modell-durchlaufs erkennen. Das Modell benötigt oft bei weitem nicht alle Epochen für eine Konvergenz. Earlystopping bricht den Vorgang bei zu wenig Veränderung verfrüht ab und ermöglicht so eine große Zeitersparnis. Darüber hinaus lässt sich ein mit Anzahl der Epochen zunehmend großer Overfitting Gap erkennen. Der Loss auf den Trainingsdaten verbessert sich vortlaufend während der Loss auf den Testdaten stagniert.

Die bayesianische Optimierung lieferte bei deutlich geringerer Rechenzeit eine vergleichbare Performance wie Random- und Grid-Search. Sie wurde im folgenen auch für die Ermittlung der Parameter der Netze mit angepassten Features eingesetzt. Die Hinzunahme der geografischen Features und Bedrooms_per_Room führte zu keiner direkten Verbesserung der Konvergenz oder des R^2 -Wertes. Die Feature-Importance-Analyse zeigte zudem, dass populationsbasier-

erkennen ist eine hohe Featureimportance der beiden eingeführten Distanz Features. Dennoch verbesserte sich der R^2 -Wert durch diese Einführung nicht unmittelbar. Ein anschließender Versuch, das Netz ohne die schwächsten Features zu trainieren, resultierte in einem verschlechterten R^2 -Wert von 0.783. Dies führte zu der Erkenntnis, dass neuronale Netze auch aus Merkmalen mit geringer Importance wertvolle Signale und Informationen ziehen, die dem Rauschen dieser Features überwiegen, weshalb keine Features beschnitten wurden. Der Vergleich zeigte auch, dass der Random Forest minimal bessere Ergebnisse ($\Delta_{Baseline} R^2 = 0.0044$) lieferte als das neuronale Netz. Mein bestes neuronales Netz mit Ensemble-Methode erreichte einen R^2 -Wert von 0.7958 auf den Testdaten (vgl.Fig. 5). Das Ziel eines R^2 -Wertes von über 0.8, das durch den Vergleich mit einem Random Forest mit gradient boosting erreicht wurde, konnte damit knapp nicht erreicht werden.

1.3.1. Vergleich mit linearer Regression

Tabelle 1. Vergleich linearer Regression mit Neuronalem Netz		
Metrik	Lineare Regression	Neuronales Netz
R^2 – Score	0.6326	0.7958 ($\Delta_{Baseline} R^2 = 0.1632$)
MAE	0.4331	0.2895
RMSE	0.5779	0.4308

Das beste in meinem Notebook (und in dem Projekt) erarbeitete Modell zeigt eine deutliche Steigerung gegenüber der Baseline. Dabei ist zu beachten, dass ein sehr simples Netz die lineare Regression schon

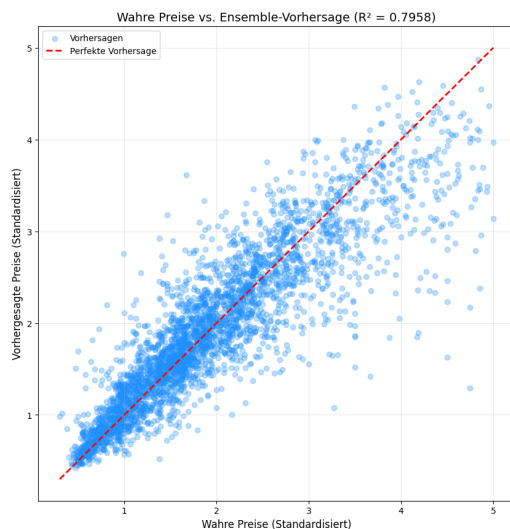


Abbildung 5. Das stärkste Neuronale Netz (siehe Vergleich mit linearer Regression)

Regression und einem neuronalen Netz. So können simple lineare Zusammenhänge direkt, ohne Verzerrungen und nichttriviale Abhängigkeiten wie zuvor durch ein komplexes Netz gelernt werden.

stark übertrifft und jedes weitere Prozent mehr Leistung sehr viel Optimierung bedurfte. Des Weiteren benötigt das Neuronale Netz sehr viel mehr Rechenzeit und verliert im Vergleich zur linearen Regression fast vollständig die Interpretierbarkeit.

1.4. Schwierigkeiten und Grenzen

1.4.1. Technische Herausforderungen

Die Implementierung der Tools wurde mehrfach durch technische Probleme gebremst. Die Installation des `keras.tuner` erforderte ein Bugfixing und ein Downgrade des Python-Environments auf Version 3.10.20. Darüber hinaus kam es wiederholt zu Git-Merge-Konflikten durch ID-Änderungen in den Jupyter Notebooks, was den Einsatz von Tools wie `nbstripout` [5] unumgänglich machte. Auch der Umgang mit der `tau-class` LaTeX-Umgebung in VS-Code stellte anfangs eine Herausforderung dar, lohnte sich aber alles in allem. Alles in allem wurden einige Stunden für das Beheben technischer Probleme benötigt.

1.4.2. Methodische Grenzen

Eine wesentliche Grenze des aktuellen Modells liegt in der Verarbeitung schiefer Datenverteilungen (z. B. bei `MedInc` und `Population`), wie in Fig. 2 zu erkennen ist. Der eingesetzte `StandardScaler` ist nicht optimal, um die Tabellendaten für das neuronale Netz vorzubereiten. Sowohl in Fig. 2 als auch in Fig. 5 ist die Schiefe der Verteilung und die mittlere Unterschätzung der Hauspreise zu erkennen.

1.5. Zwischenstand und Ausblick

Am Ende der primären Bearbeitungszeit wurde ein starkes neuronales Netz entwickelt, welches die lineare Baseline weit übertrifft. Die Erkenntnisse aus dem Feature-Engineering und den Hyperparameter-Optimierungen sind erfolgreich evaluiert worden. Aktuell liegt der Fokus auf der strukturierten Dokumentation der Ergebnisse für den Abschlussbericht. Ein vielversprechender nächster Schritt zur Überschreitung der 0.8-Grenze beim R^2 -Score wäre die Anwendung robusterer Daten-Transformationen [1], um die Datenverteilungen netzfreundlicher zu gestalten und so zu den Ensemble-Methoden der baumbasierten Modelle aufzuschließen. Die Implementierung eines `PowerTransformer` (z. B. nach Yeo und Johnson [1]) zur Symmetrisierung der Daten, wie es in der Fachliteratur für prädiktive Modellierung [2] empfohlen wird, wäre ein Ansatz für eine solche Verbesserungen des Netzes. Eine weitere Idee ist die Kombination von linearer

2. EINSCHÄTZUNG DER GRUPPENMITGLIEDER

Name: Florian Adamczyk

Florian fiel während der Projektarbeit vor allem durch seine organisatorischen Fähigkeiten und seine Hilfsbereitschaft auf. Sein strukturierter Aufbau der Arbeitsumgebung ermöglichte eine gute Übersicht, insbesondere bei der Zusammenfassung und Einordnung der Gruppenergebnisse. Zudem stand er der Gruppe stets zur Seite, wenn Probleme mit Git auftraten.

Zu Beginn gestaltete er das Projekt aufgrund seiner Erfahrung stark nach seinen eigenen Vorstellungen, sodass es zeitweise schwerfallen konnte, eigene Beiträge einzubringen. Dies änderte sich im Laufe des Projekts jedoch zunehmend, sodass die finale Gestaltung homogener aus den Beiträgen aller Teammitglieder resultierte.

Name: Felix Hollfoth

Felix zeichnete sich durch ein tiefes inhaltliches Verständnis aus, insbesondere im Bereich der neuronalen Netzarchitektur. So konnte ich durch seine Hilfe Fehler in meinem eigenen Netzaufbau ausbessern. Dabei glänzte er durch kurze und klar verständliche Hilfestellungen.

In einigen wenigen Gruppentreffen hielt er sich bei der Ideenfindung zu Themen, die nicht das ihm zugeordnete Ressort betrafen, eher zurück.

Name: Kolja Scriba

Kolja half mir und den anderen insbesondere bei grundlegenden Fragen zu derartigen Projekten sowie mit hilfreichen Kniffen. Zudem beriet er mich in Fragen zum Zeitmanagement stets zuverlässig und zeitnah.

Da wir in verschiedenen Zeiträumen an dem Projekt arbeiteten, war nicht immer unmittelbar klar, welcher Stand aktuell herrscht. Nach seiner ihm zugedachten Arbeitszeit benötigte es deshalb oft eine Einführung in die aktuellen Schritte. Anschließend half er jedoch immer anstandslos weiter.

Name: Björn Becker

Björn arbeitete am engsten mit mir zusammen und half immer bei Problemen. Die inhaltliche Absprache und die Arbeitsteilung funktionierten problemlos.

Einzig zu beanstanden ist eine Fehlabsprache zwischen ihm und Felix, die zunächst zur Auslassung der Untersuchung der optimalen Tiefe führte. Dies konnte jedoch schnell behoben und entsprechend nachgearbeitet werden.

3. ZUSAMMENFASSUNG DER ERGEBNISSE DER GRUPPE

Neuronales Netz für den kalifornischen Hauspreis-Datensatz

Felix Neumann^a, Florian Adamczyka^a, Felix Hollfoth^b, Kolja Scriba^c, Björn Becker^d and Felix Neumann^e

^aJustus Liebig Universität, Matrikelnr. 116639

^aJustus-Liebig-Universität, Matrikelnr. 8105234

^bJustus Liebig Universität, Matrikelnr. 8221324

^cJustus Liebig Universität, Matrikelnr. 8111369

^dJustus Liebig Universität, Matrikelnr. 8206290

^eJustus Liebig Universität, Matrikelnr. 8116639

Gruppe 308 – KI I Projekt WS 2025/26

Zusammenfassung—Dieser Ergebnisbericht dokumentiert die Ergebnisse der Gruppe 308 der Projektarbeit zu Maschinellem Lernen im Modul Künstliche Intelligenz 1. Die Aufgabenstellung lautete auf dem Datensatz zu den kalifornischen Hauspreisen, wie in der Vorlesung betrachtet ein neuronales Netz, um die Hauspreise zu beschreiben zu entwickeln. Im Anschluss soll dieses mit der linearen Regression verglichen werden. Die Bearbeitungszeit betrug pro Gruppenmitglied mindestens 75 Stunden und wurde in mehreren Meetings zwischen den Personen aufgeteilt. Neben der linearen Regression wurden auch andere Regressionsmodelle als Vergleichsgrundlage für das Neuronale Netz herangezogen. Der Schwerpunkt liegt allerdings auf der sukzessiven Verbesserung des Neuronalen Netzes, unter Anwendung der Vorlesungsinhalte. Die Optimierung der Regression erfolgt unter ständiger Kontrolle der Fehlergrößen wie dem Mean Squared Error (=MSE) und dem Root Mean Square Error (=RMSE), sowie dem coefficient of determination ($= R^2$) als quantitativer Vergleich der Modelle.

Keywords—neuronales Netz, California Housing, TensorFlow, Regression, Machine Learning

3.1. Explorative Datenanalyse, -Bereinigung und -Darstellung (Florian)

Die explorative Datenanalyse dient der Aufnahme von Eckdaten des Datensatzes und dem Erlangen eines Gefühls für potenzielle Besonderheiten, die in der Datenbereinigung für ein besseres Ergebnis sinnvoll behoben werden.

3.1.1. Analyse

Die Explorative Datenanalyse ergab, dass zwei der acht Features des Datensatzes cut-off-Werte besitzen. Eine unrealistisch hohe Anzahl an Werten befindet sich hierbei genau an der Grenze der aufgetragenen Werte. Zudem ist eine hohe Streuung der Werte mit einigen vermeintlichen Ausreißern direkt zu erkennen. Um damit überhaupt beginnen zu können, hat Florian ein Projektinfrastruktur mit git, allgemeinen Funktionen, Anweisungen und den utils geschaffen, sowie die Daten eingelesen. Auf dieser Grundlage konnte dann mit der inhaltlichen Arbeit begonnen werden.

3.1.2. Bereinigung

Die Cut-Off-Werte der Features 'MedHouseVal' und 'HouseAge' wurden mit den Outliern über dem 98%-Quantil bei den Features 'AveRooms', 'AveBedrms', 'Population' und 'AveOccup' entfernt, da diese Artefakte darstellen und das Rauschen des Datensatzes erhöhen. Aus den ursprünglich 20.000 Datenpunkten verbleiben nach der Bereinigung noch 17.386.

3.1.3. Skalierung

Zuletzt wurde der Datensatz noch in einen Test-(80% Anteil) und Trainingsdatensatz(20% Anteil) aufgeteilt. Aus dem Testdatensatz wurde dann zum Lernen der Modelle noch ein Validierungsdatensatz entnommen. Für das Training der Modelle mit verschiedensten Aktivierungsfunktionen ist eine Skalierung zwingend notwendig um

ein sinnvolles Ergebnis zu erhalten. In Notebook 0c wurde zu diesem Zweck ein Standard-Skalierer und drei Minimum-Maximum-Skalierer implementiert, auf die in den jeweiligen Notebooks zurück gegriffen werden wird.

3.1.4. Darstellungen bereinigter Daten

Fig. 6 Datensatz mit entfernten Cut-Off-Werten und Ausreißern sortiert nach Features. Im Vergleich mit den Originaldaten gibt es bei den modifizierten Features keine gehäuften Datenansammlungen am Ende der Skala und keine groben Ausreißer mehr.

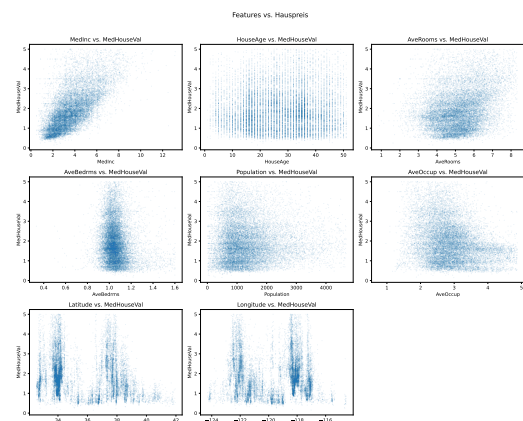


Abbildung 6. Bereinigte Daten sortiert nach Features.

Fig. 7 Die nach der Datenbereinigung durchgeführte Korrelationsanalyse zeigt den starken Zusammenhang zwischen 'MedHouseVal' und 'MedInc'.

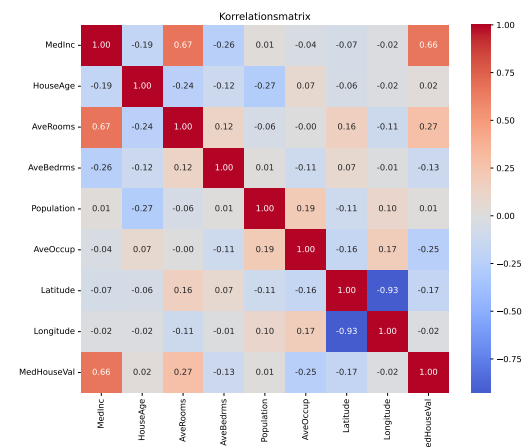


Abbildung 7. Korrelationsanalyse der bereinigten Daten

3.2. Lineare Regression

Die als Vergleichsmodell für die Bewertung komplexerer Modelle von Florian implementierte lineare Regression, erzielt unskaliert folgende Ergebnisse:

Tabelle 2. Ergebnisse der linearen Regression

Metrik	Train	Test
$R^2 - \text{Score}$	0.6465	0.6326
MAE	0.4252	0.4331
RMSE	0.5710	0.5779

Diese Werte stellen die "Baseline" für Einordnung komplexerer Modelle dar und geben einen ersten Anhaltspunkt über die potenzielle Leistung der Regressionen.

3.3. LASSO und Ridge (Florian, Kolja)

Eine Erweiterung der linearen Regression durch eine LASSO-Ridge Kombination, sowie die Verwendung des Standard-Scaler führten zunächst zu keiner Verbesserung gegenüber der Baseline. Entscheidende Unterschiede zeigten sich durch die Einführung der Polynomialen Erweiterung. Zunächst eine Verbesserung in der zweiten und nochmal geringfügig in der dritten Ordnung. Am Ende konnte ein $R^2 = 0.7184$ auf die Testdaten erreicht werden ($\Delta_{\text{Baseline}} R^2 = 8.58\%$). Durch die gewählten Methoden konnte auch ein sehr geringer Overfitting-Gap erzielt werden.

3.4. Decision Tree Regression (Florian)

Ausgehend von den Vorlesungsinhalten wurden Decision Trees zunächst als Referenz ohne Einschränkungen und dann mit Beschränkung der maximalen Tiefe gegen Overfitting Probleme trainiert. Der Overfitting-Gap bei erstem lag bei knapp 45%. Bei zweitem wurde der bias-varianz-tradeoff sichtbar. Durch Pruning konnte das Overfitting des Baums auf die Testdaten sehr erfolgreich unterbunden werden. Wie auch bei der Explorativen Datenanalyse zeigt sich, dass das Median Einkommen (=MedInc) die Featureimportance dominiert (siehe Fig. 10).

3.5. Ensemble (Florian)

Die Ensemble-Methoden wurden als Test für die mögliche Genauigkeit der Regression verwendet. Die Ergebnisse der Decision Trees konnten als Random Forest enorm verbessert werden. Auch die Verwendung des Gradient Boosting und die Optimierung der learning rate konnten bessere Regressionen liefern. Diese Erkenntnis und ein maximales $R^2 = 0.81$ ($\Delta_{\text{Baseline}} R^2 = 17.74\%$), als Marker für die mögliche Leistung der Regression, wurden in das Training neuronaler Netze einbezogen.

3.6. K-Nearest Neighbors Regression (Florian)

Die K-Nearest Neighbors Regression hingegen zeigte keine allzu zuverlässigen Ergebnisse. Sie zeigte jedoch auf, wie unerlässlich eine Skalierung der Daten sein kann. Eine gleichzeitige Reduktion zweier Fehlerquellen führt zum bias-variance-tradeoff bei der Wahl des besten k. Durch Distance weighting wurde eine Verbesserung erzielt, allerdings rechtfertigen die Ergebnisse ($\Delta_{\text{Baseline}} R^2 \approx 3.01\%$) die viel längere Rechenzeit nicht.

3.7. Neuronale Netze

Verschiedene Neuronale Netze (=NN) wurden, ausgehend von einem sequenziellen Netz mit einem Hidden Layer (=HL) als Minimalnetz, zunächst ohne optimierte Parametrisierung, ausprobiert. Bereits bei diesem Minimalnetz ergab sich ($\Delta_{\text{Baseline}} R^2 \approx 10\%$). Im Anschluss testeten wir verschiedene Optimierungsansätze:

3.7.1. Architektur

Durch eine geschickte Wahl oder bewusstes Freistellen der Wahl der Parameter des Aufbaus des neuronalen Netzes wurden kleine Verbesserungen der Regression erreicht.

Layer (Björn, Felix H., Felix N., Florian; Kolja) Die Variation der Breite und Layer ergab keine großen Veränderungen für das Neuronale Netz. Ein zu schmaler Layer vermag die komplexen Zusammenhänge zwischen einzelnen Merkmalen nicht ausreichend zu beschreiben. Zu viele Neuronen in einem Layer führten hingegen zu Overfitting. Dieses konnte durch die Verwendung eines Dropoutlayers drastisch reduziert werden. Es stellte sich zudem heraus, dass obgleich bei isolierter Analyse der Tiefe an einfachen Netzen eine größere Tiefe den R^2 weiter steigen lässt, bevor bei ungefähr 10 Layern das overfitting übernimmt vgl. Fig. 8, dass bei komplexeren und breiteren Modellen jedoch keine große Tiefe für eine relativ genaue Regression notwendig ist. Diese erhöht nur zunehmend das Overfitting. Es wurden gute Ergebnisse mit zwei-drei Hidden Layers, einer [128 – 64 – 32]-Neuronenverteilung und einem Dropoutlayer, erzielt.

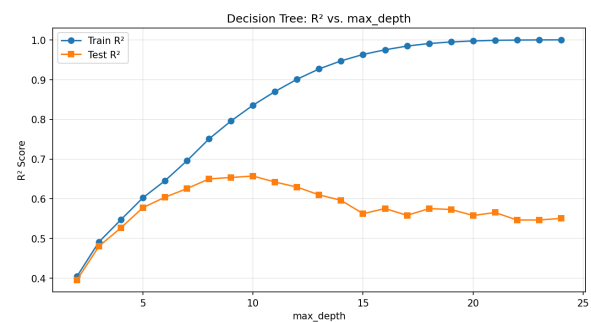


Abbildung 8. maximale Tiefe

Regularisierung (Kolja) Untersuchungen des Einflusses einer L2-Regularisierung verglichen mit dem Basismodell ohne Regularisierung erzielte einen signifikant geringeres Overfitting-Gap (-40%). Damit verbessert sich die Generalisierungsfähigkeit der Modelle erheblich. Nach Analyse der L2-Regularisierung, des Dropout-Verfahrens und einer Kombination konnte ein von Modell zu Modell sukzessive verringertes Overfitting zulasten des R^2 festgestellt werden. Durch die drastische Verringerung des Overfittinggaps reduzierte sich dieser um einige Zehntel Prozent.

Aktivierungsfunktionen (Felix H., Kolja) Ein Test der sechs gängigsten Aktivierungsfunktionen (ReLU, tanh, sigmoid, ELU, SELU, Leaky-ReLU) im Zusammenspiel mit den verschiedenen Skalierung zeigt die beste Leistung auf den Testdaten mit dem Tangens hyperbolicus angewandt auf den Standard-Scaler. Dieser erzeugt viele Daten um Null herum, die tangens hyperbolicus gut verarbeiten kann. Zudem neigt diese Funktion wegen ihrer Glätte zu wenig Overfitting. Spätere komplexere Modelle zeigen, dass die Leistung nur geringfügig von der Aktivierungsfunktion abhängt und die Wahl dem Modell überlassen werden kann.

Hyperparameter (Björn, Felix H., Felix N.) Zunächst fand eine Analyse der Hyperparameter unter festhalten der restlichen Parameter statt, um eine Idee für die Größenordnung zu gewinnen. Die isolierte Hyperparameterbetrachtung endete dabei häufig in lokalen Minima ohne eindeutiges Optimum. Dennoch stellten sich folgende Größenordnungen als Anhaltspunkte für weitere Untersuchungen heraus.

Tabelle 3. Ergebnisse der Hyperparameteruntersuchung

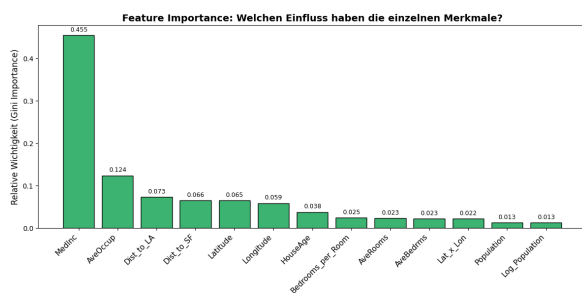
Hyperparameter	Größenordnung	Kommentar
Epochen	300-500	
learning-rate	0.01- 0.001	im Folgenden meist mit Lr-Scheduler. Dennoch fast immer in diesem Bereich
Batch-Size	16-128	größer würde zu einer zu langen Konvergenzzeit führen

Das Zusammenspiel der Hyperparameter wurde zunächst mit Random Search und zur höheren Recheneffizienz mit einer Bayesschen Optimierungsfunktion durchgeführt. Die Learningrateoptimierung erfolgt dynamisch während des Trainings mit einem Learningrate-Scheduler.

Iles in allem führt die händische Veränderung der Architektur, der Lernrate und der Epochen zunächst zwar zu kleinen Ergebnisschwankungen, ist aber ohne systematisches Vorgehen und Simultanoptimierung nicht zielführend. Die besten Modelle variieren daher in der Zusammensetzung der Architektur, liegen aber alle in den genannten Größenordnungen.

3.7.2. Feature engineering (Felix N., Björn, Felix H.)

Eine Untersuchung der Features zeigt eine deutliche Spanne an potenziellen Informationsgewinn durch die einzelnen Features. Daher wurde versucht die Modelle mit den wichtigsten vier bis fünf und auch nur dem wichtigsten Feature zu trainieren. Eine geringere Featureanzahl verringert die Komplexität des Modells und somit die Rechenzeit. Obgleich eine geringere Featureanzahl zu keine höheren Leistung führt, können Ensembles aus Modellen die verschiedenen viele Features berücksichtigen dies leisten. Um die Korrelation einzelner Merkmale untereinander zu nutzen wurden weitere Features aus dem Datensatz zusammengefasst. Die so neu entstandenen Features sind „Bedrooms-per-Room“, „Dist-to-SF“ und „Dist-to-LA“, als Hypothese einer Preissteigerung in Abhängigkeit der relativen Nähe zu den größten Ballungsräumen, sowie „Lat-x-Lon“, als allgemeine geografische Einordnung. Aufgrund der Schwierigkeiten mit schiefen Verteilungen wurde zudem die Population in logarithmischer Form eingebracht. Auch die Einführung dieser Features brachte keine nennenswerte Leistungssteigerung (ca.0.5%) oder Fehlerminderung mit sich. Dennoch haben vor allem die beiden Distance-Features einen Mehrwert für das Projekt, da ihre Feature Importance relativ hoch ist. Wie im Plot Fig. 9 zu sehen, haben die Distancefeatures keine unwesentliche Feature Importance. Das Median Einkommen bleibt jedoch das stärkste Feature für den Datensatz.

**Abbildung 9.** Featureimportance

Obgleich vorallem die Distance-Features keine unwesentliche Feature Importance aufweisen, bleibt das Median Einkommen das stärkste Feature für den Datensatz.

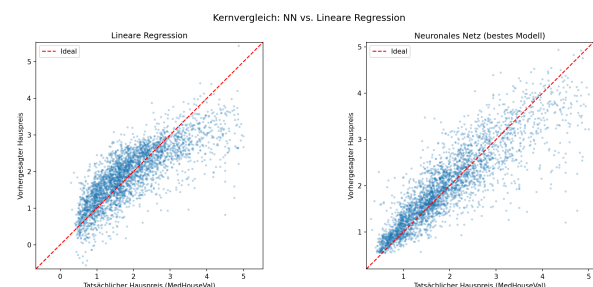
3.7.3. Gesamtvergleich mit der Linearen Regression

Der Gesamtvergleich zwischen linearer Regression und unserem stärksten Neuronalen Netz zeigt eine deutliche Leistungsdifferenz:

Tabelle 4. Vergleich linearer Regression mit Neuronalem Netz

Metrik	Lineare Regression	Neuronales Netz
$R^2 - Score$	0.6326	0.7958 ($\Delta_{Baseline} R^2 = 0.1632$)
MAE	0.4331	0.2895
RMSE	0.5779	0.4308

Der direkte Vergleich zwischen linearer Regression und neuronalen Netzen offenbart ein klassisches Spannungsfeld in der Datenwissenschaft: die Abwägung zwischen Vorhersagegenauigkeit und Interpretierbarkeit. Da bereits ein einfaches neuronales Netz als Minimalnetz einen deutlich besseren Score auf den Testdaten erzielt als die lineare Regression vgl. Fig. 10, lässt sich schließen, dass die Features und ihr Zusammenwirken komplexe, nichtlineare Zusammenhänge aufweisen. Während neuronale Netze diese abbilden können, bedarf es bei der linearen Regression aufwendigen Feature-Engineering, wie etwa durch polynomiale Erweiterungen.

**Abbildung 10.** Lineare Regression vgl. einfaches Neuronales Netz

Das Training eines NN benötigt ein Vielfaches an Rechenzeit und benötigt ein umfangreiches Hyperparameter-Tuning, unter anderem bezüglich der Architektur, Lernrate, Dropout-Rate und Batch-Größe für ein optimales Ergebnis. Wie die Untersuchungen zeigten, neigt beispielsweise ein unreguliertes Modell mit einer [128-64-32]-Neuronenverteilung zu starkem Overfitting. Die Generalisierungsfähigkeit kann jedoch durch den Einsatz von Regularisierung und Dropout erheblich robuster gestaltet werden.

Zudem fehlt den neuronalen Netzen die Interpretierbarkeit fast vollständig. Die lineare Regression hingegen überzeugt durch ihre leicht nachvollziehbaren Koeffizienten.

Die nicht-linearen Zusammenhänge des kalifornischen Hauspreis Datensatzes lassen sich sehr gut durch ein neuronales Netz beschreiben. Das Optimalergebnis des Random Forest mit Gradient Boosting wurde jedoch noch nicht erreicht. Mögliche Ansätze für noch weitere Optimierungen wäre zum Beispiel der Einsatz von Batch Normalization zur Stabilisierung des Lernprozesses oder die Verwendung eines Powertransformers. Ansätze wie die bereits untersuchte manuelle Logarithmisierung der Population könnten damit automatisiert werden. Dass dies zu Lasten der Interpretierbarkeit geht, wöge im Vergleich zur linearen Regression nicht weiter auf.

4. PROTOKOLL DER GRUPPENTREFFEN

Gruppentreffen 1

Datum: 11.02.2026

Teilnehmende: Florian, Kolja, Felix N., Felix H., Björn

Zusammenfassung und inhaltlicher Verlauf:

- Die Gruppe lernte sich kennen und klärte die technische Organisation, wie die Nutzung von Python 3.14.3, einem GitHub-Education Account und einem Git-Repository.
- Felix Neumann wurde zum Gruppensprecher bestimmt.
- Es wurde festgelegt, funktional zu programmieren, um die Übersichtlichkeit zu wahren.

Aufgabenverteilung und zeitliche Absprachen:

- Florian übernahm die Bereinigung des Datensatzes unter Einbezug von Vorlesungsinformationen.
- Als zeitliche Absprache für das nächste Treffen wurde der 17.02.2026 vereinbart.

Gruppentreffen 2

Datum: 17.02.2026

Teilnehmende: Björn, Kolja, Florian, Felix H.

Zusammenfassung und inhaltlicher Verlauf:

- Die Nutzung von notebook-stripout und VS Code mit GitHub wurde besprochen und eingeführt.
- Eine Idee zur Nutzung des JupyterHubs der JLU für größere Berechnungen wurde zunächst auf spätere Meetings (beim Modelltraining) verschoben.
- Die Gruppe einigte sich auf eine einheitliche Beschriftung des Codes per Kommentar oder Markdown.

Aufgabenverteilung und zeitliche Absprachen:

- Florian und Kolja wurden mit der Datenbereinigung und Datenanalyse betraut.
- Felix übernahm die Aufgabe der Datentransformation.
- Das nächste Treffen wurde für den 27.02.2026 auf Discord angesetzt.

Gruppentreffen 3

Datum: 20.02.2026

Teilnehmende: Kolja, Florian

Zusammenfassung und inhaltlicher Verlauf:

- Bisherige Arbeitsschritte sowie die Installation von Python 3.11 und 3.13 wurden besprochen.
- Es traten Probleme mit der Python-Version auf einem Intel-Mac in Verbindung mit TensorFlow auf, deren Lösung diskutiert wurde.
- Die Projektsortierung und eine einheitliche Code-Beschriftung durch Markdown wurden weiter vorangetrieben.

Aufgabenverteilung und zeitliche Absprachen:

- Die Aufgaben aus dem letzten Treffen für Florian, Kolja (Datenbereinigung/-analyse) und Felix (Datentransformation) blieben bestehen.
- Kolja übernahm zusätzlich die Lösung des TensorFlow-Problems.
- Der Termin für das nächste Treffen (27.02.2026) wurde bestätigt.

Gruppentreffen 4

Datum: 27.02.2026

Teilnehmende: Florian, Kolja, Felix N., Felix H., Björn

Zusammenfassung und inhaltlicher Verlauf (Planungsänderungen):

- **Anpassung des Vorgehens:** Es wurde festgestellt, dass die bisherige Arbeit teilweise an der Aufgabenstellung vorbeiging, weshalb ein stärkerer Fokus auf Neuronale Netzwerke gelegt wurde.
- Trotz anhaltender Versionsprobleme wurde beschlossen, bei TensorFlow und inhomogenen Python-Versionen zu bleiben.
- Zur besseren Übersicht wurde vereinbart, bei der Arbeit mit Hyperparametern und großen Veränderungen künftig Git-Banches zu nutzen.

Aufgabenverteilung und zeitliche Absprachen:

- Felix: Prüfung verschiedener Aktivierungsfunktionen, Lernrate, Batchsize und Epochenanzahl.
- Kolja: Untersuchung verschiedener Lossfunktionen sowie der Anzahl und Breite der Hidden Layers.
- Florian: Zurücknahme in den Aufgaben aufgrund eines bisher sehr großen Workloads.
- Die endgültige Abgabe wurde auf spätestens Gründonnerstag terminiert, das nächste Treffen auf den 17.03.2026.

Gruppentreffen 5

Datum: 17.03.2026

Teilnehmende: Florian, Kolja, Felix N., Felix H., Björn

Zusammenfassung und inhaltlicher Verlauf:

- Kolja und Felix H. stellten ihre Projekte vor. Die Erkenntnisse von Felix H. zeigten, dass eine Beschränkung auf bestimmte Features bessere Ergebnisse liefert, da einige Features zu starkes Rauschen verursachen.
- Aufgrund zu großer Variation bei erneutem Ausführen wurde festgestellt, dass Ensembles eine gute Möglichkeit zur Score-Verbesserung sind.
- Die Gruppe einigte sich auf den Harvard-Zitierstil und die Erstellung einer LaTeX-Vorlage für die Gruppenergebnisse.

Aufgabenverteilung und zeitliche Absprachen:

- Person 1: Systematische Hyperparameter-Optimierung durchführen und analysieren.
- Person 2: Modellvergleich durch Implementierung alternativer Modelle (Random Forest, Gradient Boosting) im Vergleich zu neuronalen Netzen durchführen.
- Björn und Felix wurden beauftragt, die Ergebnisse zusammenzufassen und im Code verständlich zu erläutern.

Gruppentreffen 6

Datum: 26.03.2026

Teilnehmende: Florian, Kolja, Felix N., Felix H., Björn

Zusammenfassung und inhaltlicher Verlauf:

- Felix N. und Björn stellten ihre Projekte vor.
- Die Gruppe einigte sich auf verschiedene Formatierungen und beschloss, nbstripout zu deinstallieren, damit die Notebooks ausgeführt gepusht und anschließend vom Gruppensprecher gezippt werden können.
- Wichtige Grafiken sollen ins Logbuch eingefügt werden; zudem fand eine kurze Mediation statt.

Aufgabenverteilung und zeitliche Absprachen:

- Felix N. schreibt mit Unterstützung der Gruppe die Gruppenergebnisse zu Ende.
- Aufgrund von Urlaub einigte sich die Gruppe auf eine zeitnahe Abgabe.

5. LOGBUCH

Name: Felix Neumann **Gruppe:** 308

Zeitraum: 20.02.2026 – 15.04.2026

Eintrag 1

Datum: 20.02.2026 **Titel:** Projektstart — Aufgabe lesen, Repository einrichten

Aus Gruppentreffen: [1] (20.02.2026) (ca. 2 h)

Arbeitsschritte:

- Aufgabenstellung gelesen und verstanden
 - Repository geklont, Packages installiert (`pip install -r requirements.txt`)
 - `nbstripout --install` eingerichtet
 - Erste Übersicht über den California Housing Datensatz verschafft (`fetch_california_housing()`, 8 Features, Zielvariable: `MedHouseVal`)
 - Besprechung allgemeines Vorgehen
 - Einlesen in verschiedene Formen des NN
 - verknüpfen Vorlesungsinhalte mit Implimentationen im Notebook
 - einrichten Github zum Austausch der Inhalte
-

Eintrag 2 (ca. 6 h)

Datum: 17.03.2026 **Titel:** [Allgemeine Einarbeitung]

Aus Gruppentreffen: [4, 5]

Arbeitsschritte:

- Vorbereitung VS-Code
- Versuch Codeausführung mit bereits eingerichtetem KI1-Environment
- Neuinitialisierung Python environment
 - Pakete installiert
 - Problemlösen Tensorflow
- Einlesen in neuste Änderungen der Gruppenmitglieder
- Aufarbeitung Vorlesungsinhalte
- einarbeitung in Git
- Gruppentreffen 5 (siehe Gruppentreffen)
 - Absprache zur Zeitaufteilung und Bearbeitung der NN
- logbuch angelegt
- Recherche allgemein zu vorgehen beim Aufbau neuronaler Netze

- Ausarbeitung zu Ideen zu:
 - random search, grid search, Datentransformation, drop out und weigth decay
- Einlesen in Fachliteratur, insbesondere: [Stuart J. Russell, Peter Norvig (2020-4th Ed.): Artificial Intelligence: A Modern Approach, Prentice Hall] **Entscheidungen:**
- Python 3.11.15 verwenden
- Absprache mit Björn zur Zusammenfassung und weiteren Optimierung der NN von Kolja und Felix H.

Schwierigkeiten / offene Fragen:

- Python environment (gelöst)

Eintrag 3 (ca. 8h)

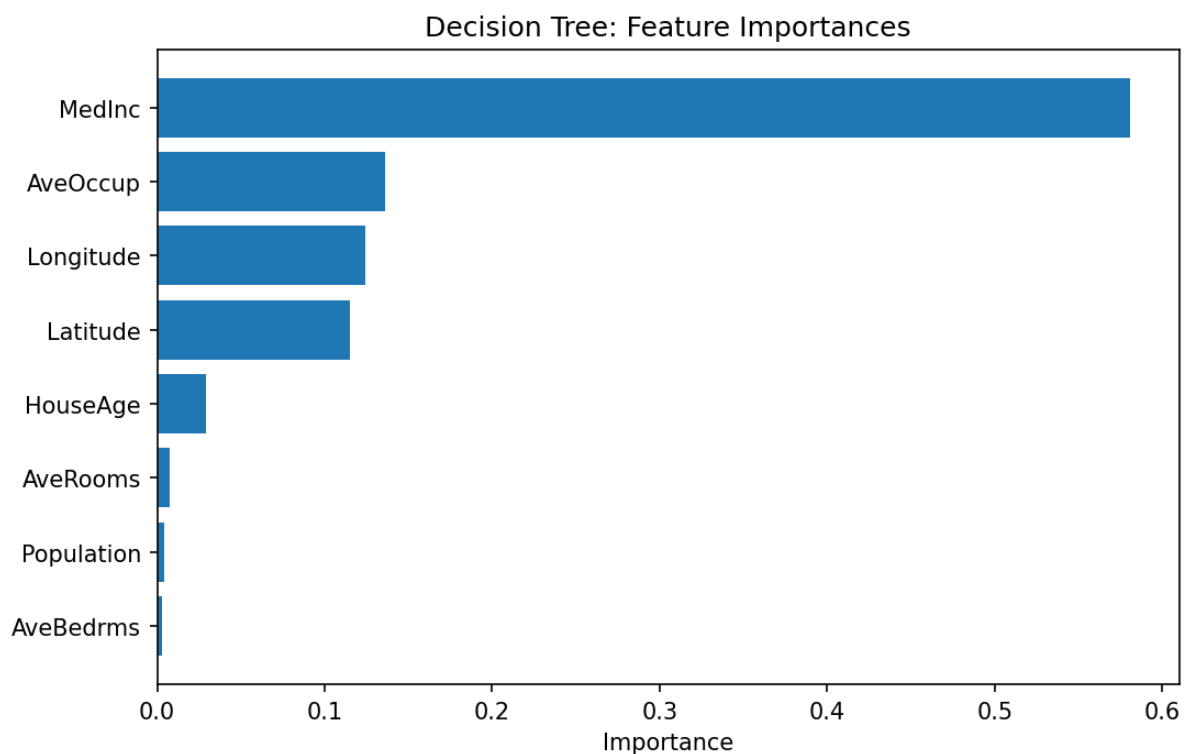
Datum: [19.03.2026]

Titel: [begin Bayesianische Optimierung]

Aus Gruppentreffen: [5]

Arbeitsschritte:

- Zusammenfassung NN Felix H. für mein Vorgehen
 - Konzentration auf Features nach Einfluss auf das Modell (siehe Abbildung Feature importances)



- - durch decisiontree herausgearbeiteter Einfluss der Features bei Bearbeitung berücksichtigen
- Im Sinne der Konvergenzzeit auf die wichtigsten Features zur Beschreibung des Datensatzes nutzen
 - gegebenenfalls einzelne dieser Features zusammenfassen und weiter experimentieren

- Suche nach Optimierung mit grid- und random search
 - Erkenntnis: Felix H. schon sehr viel inklusive ensemble ausprobiert
 - effizientere Suche durch Bayesianische Optimierung
 - Idee dabei: durch geschickte Beobachtung des Lernvorganges abschätzen, ob dieser zielführend ist, oder nicht
 - gegebenenfalls wird nach weniger als der maximalen Epochenzahl abgebrochen, um Rechenzeit zu sparen
- Begin Implimentierung mit Keras tuner model
- Quelle: Luca Invernizzi, James Long, Francois Chollet, Tom O'Malley, Haifeng Jin (2019): Getting started with KerasTuner https://keras.io/keras_tuner/getting_started/
 - [build_tuner_model()] erstellt ein sequenzielles Model über Keras-Paket: "https://keras.io/guides/sequential_model/"
 - zunächst einfaches Modell zur sauberen Implimentierung der Bayes Idee
- direkte Einbindung des Dropout (aus Erkenntnissen der anderen NN)
- im Sinne des vanishing gradient und der gefahr des overfittings zunächst kein sehr tiefes Netz
- zweiter Versuch mit mehr zugelassener Breite (bis 256 Neuronen pro Layer)
- bei NN- Architektur an Erkenntnissen aus 05_Neural_Network_Felix.ipynb orientieren
 - geringe Tiefe, variable Breite
 - start learning rate 0.001
 - Verwendung der vorbereiteten Datenbereinigungen und Skalierungen [load_and_clean_data()]
- Interpretation Residuenhistogramme: (alle ähnliche Verteilung)
- Das Residuendiagramm zeigt eine Zentrierung der Werte um die Nulllinie. Das Modell hat demnach keinen systematischen Bias und unterschätzt Werte ähnlich oft, wie es sie überschätzt. Obgleich der Großteil des Fehlers in einem engen Bereich liegt, zeigt er keine perfekte Normalverteilung, sondern hat "heavy tails". Eine solche leicht Rechtsschiefe Verteilung bedeutet, dass die Preise der Häuser im Zweifel leicht unterschätzt werden. Es sind auch einige Outlier (Residuen>2) zu erkennen deren Preis massiv unterschätzt wurde. Dies könnte an dem Fehlen von Features liegen, welche diese Preissteigungen erklären könnten.

Entscheidungen:

- Pythonumgebung auf 3.10.20 für gute tensorflowpermoance downgraden
- bayesianische Optimierung zunächst einfach mit sequenziellem Modell

Schwierigkeiten / offene Fragen:

- Probleme beim installieren von keras.tuner
 - 1,5h bugfixing und zweifach neues Aufsetzen des Environments
 - downgrade auf python 3.10.20 half

Eintrag 4 (ca. 8h)

Datum: [20.03.2026]

Titel: [fortsetzung Implimentierung Bayesianische Optimierung]

Aus Gruppentreffen: [5]

Arbeitsschritte:

- gestrige Idee weiter implimentiert
- Entscheidung für optimizer adam
 - "generell zunächst basis Einstellungen Nutzen und dann komplexeres ausprobieren"
- Möglichkeit zu hinzufügen eines weiteren Layers einprogrammiert
- Probleme beim einbau des Learning rate Scheduler (*gelöst)
 - nun dynamische Anpassung der learning rate (Standartvorgehen)-> führt zu Performancesteigerung
- Implimentierung des Ensembles. auch zuvor schon deutliche verbesserung gegenüber Einzelmodel
 - Nun sowohl stärkstes Einzelmodell, als auch ensemble der stärksten Modelle im Vergleich
- hinzufügen num_initial_points=5 in tuner
- Hinzufügen "Bedrooms_per_Room" als Feature in eingelesenen Daten
- vorläufig bestes Ergebnis mit R^2 -score= 0.7925 **Entscheidungen:**
- adam optimizer
- weitere Experimente mit Optimizer zu späterem Zeitpunkt
- neues Feature durch hinzufügen: "Bedrooms_per_Room" = "AveBedrms"/"AveRooms"
 - zunächst keine Verbesserung
- Versuch durch erneute Reduktion der Layer und Erhöhung der Breite auf ≤ 512 Neuronen Performance zu erhöhen
 - $R^2 = 0.7897$
- hinzufügen leaky_relu zu möglichen Aktivierungsfunktionen
 - leaky relu setzt Werte <0 im Gegensatz zu Relu nicht auf Null, sodass diese oftmals "absterben" (Dying Relu Problem)
- Werte mit sehr geringer Steigung kommen hie rnoch durch und der Informationsfluss bleibt bestehen
- Schwierigkeiten / offene Fragen:**
- Implimentierung des LR-Scheduler (*gelöst)
- R^2 über 0.8 mit diesen Mitteln erreichbar?
- Verschiebung des Speicherortes hochwertiger Modelle

Erkenntnis:

- alles in allem gleiche performance wie bei random- und grid search mit drastisch reduzierter rechenzeit
- kein besserer R^2 durch hinzufügen eines weiteren features
- ebenfalls keine Verbesserung durch Berücksichtigung Leaky elu
-

Eintrag 5 (ca. 45h)

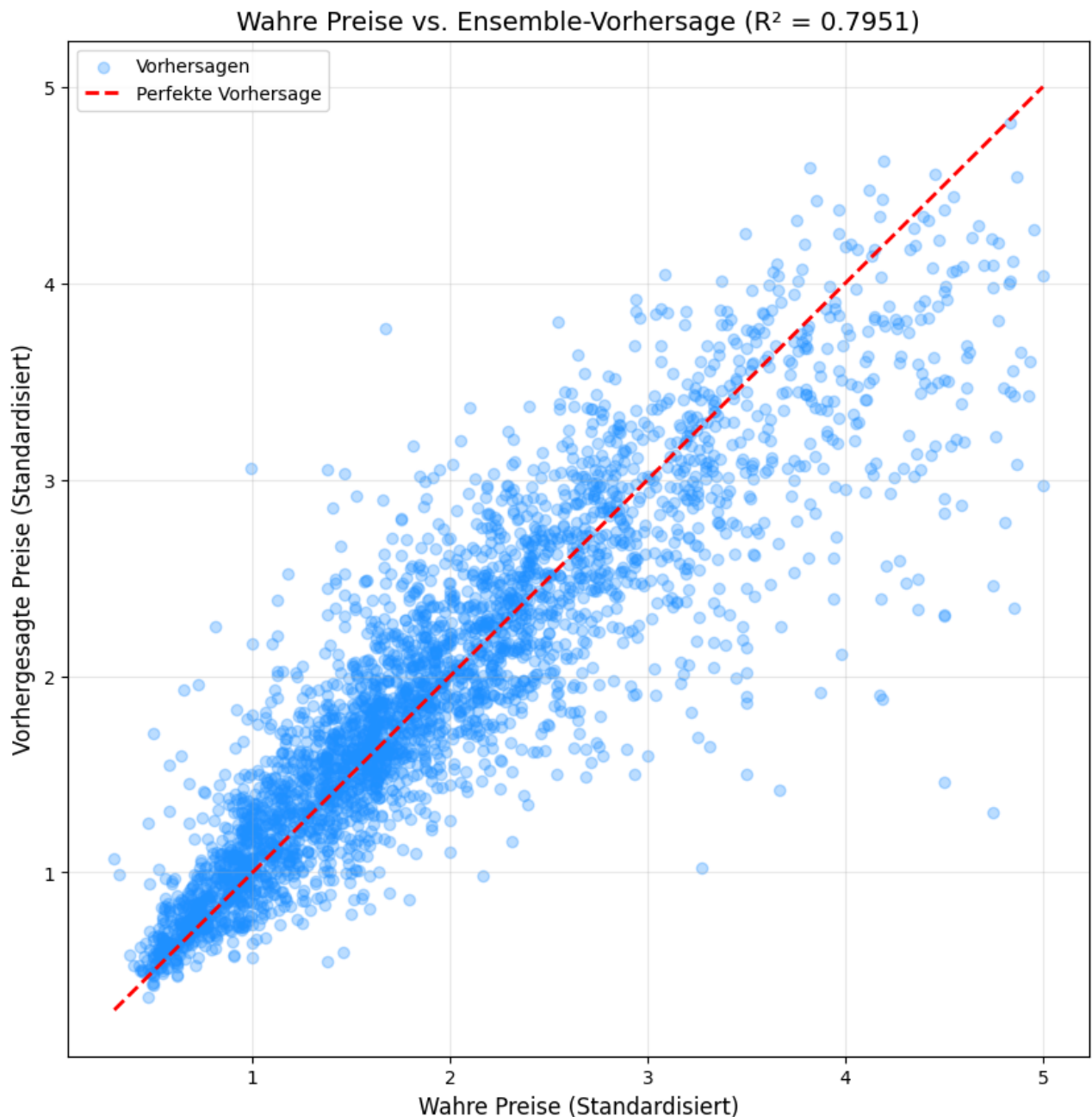
Datum: [21.03.2026]

Titel: [neue geologische Features]

Aus Gruppentreffen: [5]

Arbeitsschritte: Experiment: Implimentierung der Features 'Lat_x_Lon', 'Dist_to_SF', 'Dist_to_LA'

- Koordinaten in Longitude und Latitude als Kombinations
 - Die Idee dahinter: Aus eigener Erfahrung bekannt, dass LA und SF absolut größte Zentren und entscheidend für Wirtschaft in Kalifornien
 - Eine Orientierung in Abhängigkeit der Distanz zu beiden Zentren könnte helfen Preise exakter zu beschreiben
- Ziel: $R^2 \geq 0.8$ (Berücksichtigung des Loss und der Residuen)
 - Betrachtung darf nie nur im Sinne des R^2 getroffen werden
- Resultat: nach zwei durchläufen keine Erkennbare Verbesserung (Ziel vorest verfehlt)
- Bestes Ergebnis bisher:



Entscheidungen: Keine Blinde Untersuchung mit neuen Features mehr. AEs wird auch mit den neuen features ein random forest zur Untersuchung der feature importance durchgeführt.

Erkenntnis: Auch die Einführung der geografischen Features, bringt keine direkte Verbesserung des Modells hinsichtlich der Konvergenz und

des R^2 -Values. Das impliziert jedoch keine Nutzlosigkeit für das Modell insgesamt, da auch kein Nachteil festgestellt werden kann.

Eintrag 6 (ca. 6h)

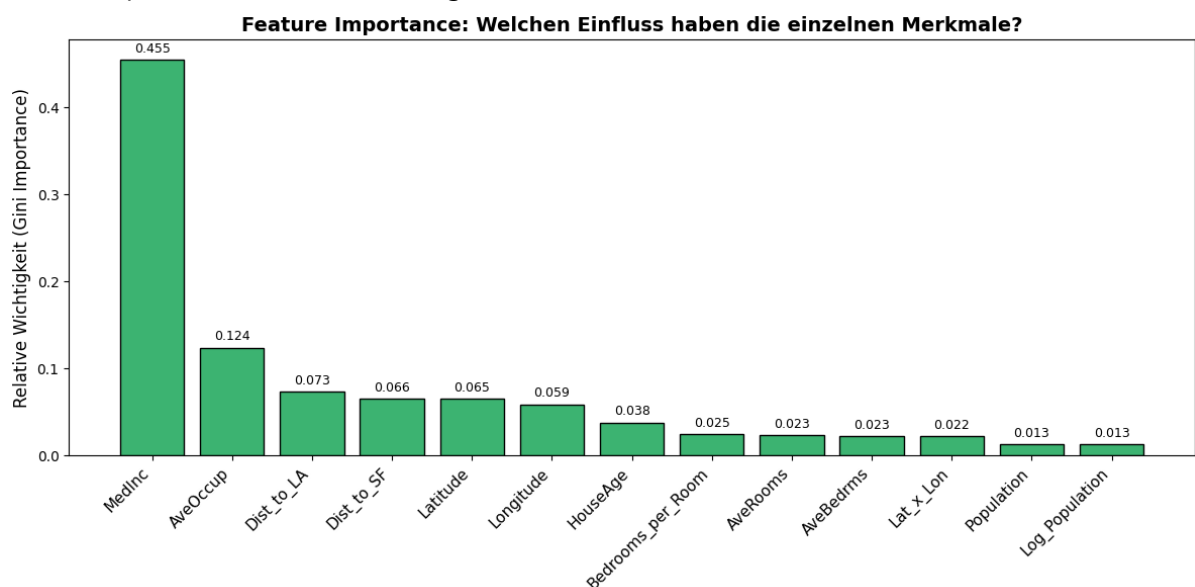
Datum: [22.03.2026]

Titel: [Random Forest]

Aus Gruppentreffen: [5]

Arbeitsschritte:

- Wie in Gruppentreffen 5 besprochen Random Forest zur feature importance Analyse
- Random Forest Zelle schreiben
 - `scikit.learn RandomForestRegressor`
 - Zunächst sicherstellen, dass `y_train` richtige size hat
 - Nutzung von `[isinstance()]`
 - `n_jobs=-1` beschleunigt Rechnung durch Nutzung aller zur Verfügung stehender Kerne
 - Skalierung der Daten für Random Forest irrelevant
 - Darstellung der importances durch Balkendiagramm **Erkenntnis:** Untersuchung der Featureimportance (zu sehen auf folgendem Plot:



) zeigt auf, dass population und auch die zuvor etwas vielversprechender wirkende `log_population`, wenig Einfluss auf das NN haben. Es kann davon ausgegangen werden, dass ihr Rauschen die positiven Effekte aufhebt.

Entscheidungen:

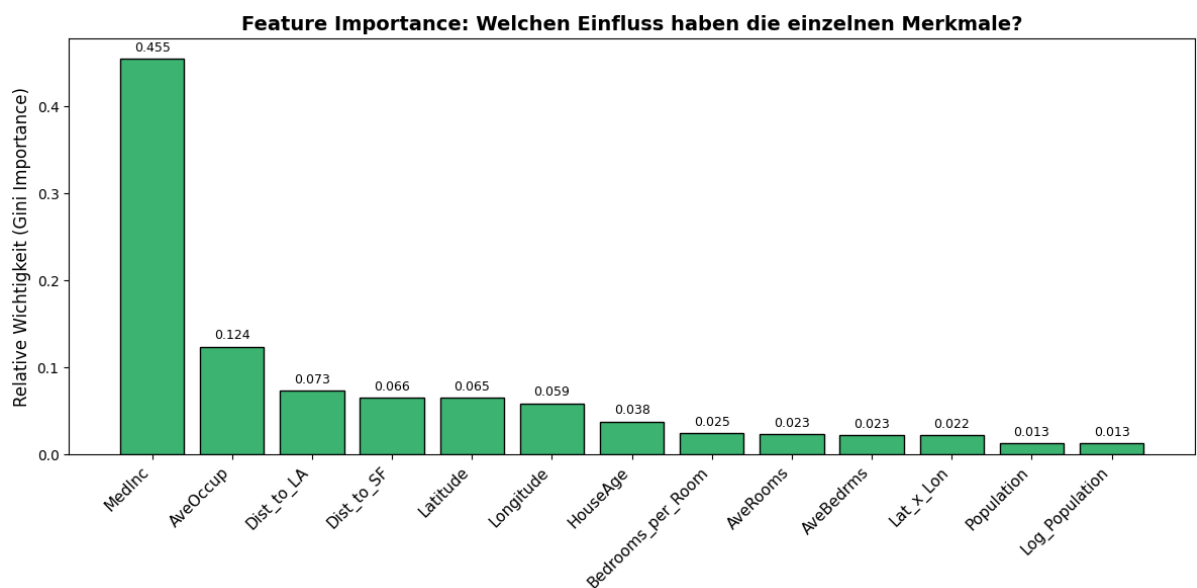
- NN ohne Population trainieren **Schwierigkeiten / offene Fragen:**
- Zunächst Fehler der eingesetzten Daten aufgrund deren Umfangs. Gelöst durch erneutes Laden der Daten (manchmal kann es einfach sein)
- Berechnungszeit vor `n_jobs` deutlich länger

Eintrag 7 (ca. 8h)

Datum: 23.03.2026 **Titel:** Interpretation bisheriger Erkenntnisse **Aus Gruppentreffen:** [5]

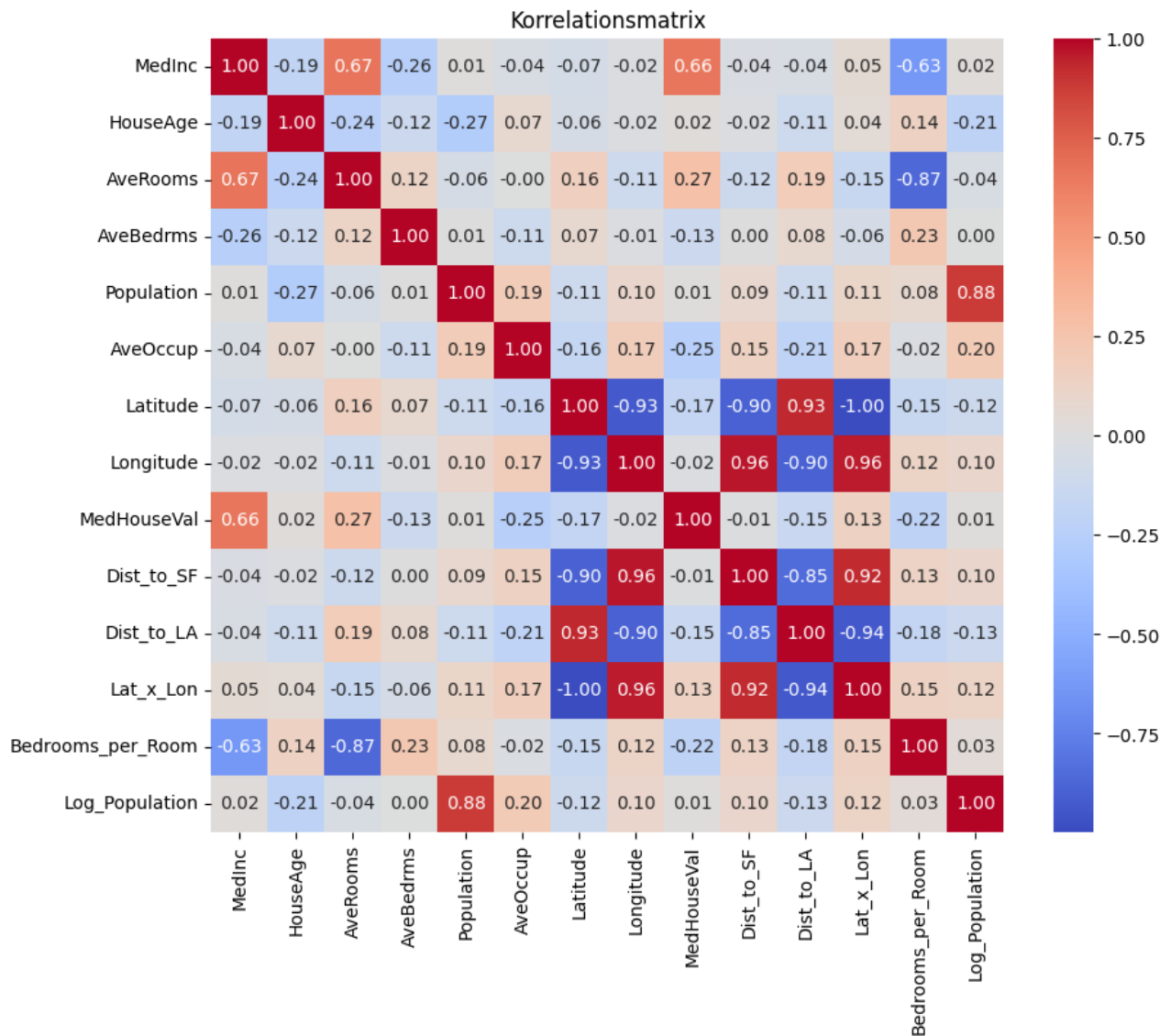
Arbeitsschritte:

- Die gestrige Entscheidung das NN ohne die Poplationsfeatures zu trainieren, da diese dem Random Forest zur Folge am schwächsten abschneiden, wurde ausgeführt
 - Das ernüchternde Ergebnis: zwar ist das Netz deutlich schneller konvergiert, doch lag der R^2 "nur" noch bei circa 0.783.
- KI Prompt: "Ab welcher Gini Importance lohnt es ein Feature für das Training eines Neuronalen Netzes zu vernachlässigen?"
 - Das Sprachmodell Gemini erklärt, dass bei einer Feature importance von deutlich unter einem Prozent das Rauschen dem Informationsgewinn überwiegt.
 - Aus dem Ergebnissen der Feature Importance analyse (



) folgt, dass einige Features zwar einen relativ geringen, in Summe aber nicht zzu vernachlässigenden Beitrag liefern. Neuronale Netze sind in der Lage auch aus kleinen Signalen Muster zu filtern und profitieren hierbei auch von solch Signalen, insbesondere wenn es um die letzten Performanceprozent geht.

- Die geringe Hilfe der neuen Features wird auch durch eine Betrachtung der Korrelationsanalyse ersichtlich:



- Besprechung mit Björn
 - Besprechung der bisherigen Ergebnisse
 - Erkenntnisgewinn aus Training der neuronalen Netze ausgetauscht
 - mit Stand der anderen Gruppenmitglieder verglichen
 - Aufteilung des weiteren Vorgehens
 - Wer fasst welches Notebook zusammen?
 - Besprechung der Ergebnisse bei Gruppentreffen am 26.03
- Implimentierung einer Latex Arbeitsumgebung in VS-Code
 - installation der notwendigen Pakete
 - einarbeiten in die Tau-class
 - Modifikation des Gitignore
- Weitere Durchläufe unter Variation der Umfänge für die Hyperparamter durchgeführt...
 - keine Nennenswerten Veränderungen gefunden
 - Ensembleperformance relativ konstant

- Random Forest liefert minimal bessere Ergebnisse als NN (Diff. $R^2 = 0.0044$).
 - Erklärungsansatz: Tabellarische Daten mit Unregelmässigkeiten, harten Kanten und schiefen Verteilungen lassen sich oft mit Entscheidungsbaum beschreiben. Neuronale Netze performen besser auf "glatten" Daten / Normalverteilungen.
 - Lösungsansätze:
 - Noch weiter Verbesserte Datenaufbereitung. Beispielweise Versuch weitere Ausreißer an der Definitionsgrenze heraus zu filtern.
 - Statt Standard-scaler oder MinMax-Scaler Powertransformer nutzen
 - Powertransformer ist stärker auf schiefen Datenverteilungen wie zum Beispiel bei MedInc und Population als StandardScaler.
 - Die Anwendung eines Powertransformer wird insbesondere für den Kaliforniahousing Datensatz empfohlen.
 - Yeo, I. K., & Johnson, R. A. (2000). A new family of power transformations to improve normality or symmetry. *Biometrika*, 87(4), 954-959
 - Kuhn, M., & Johnson, K. (2013). *Applied Predictive Modeling*. Springer. (Kapitel 3: Data Pre-processing)
 - Deshalb random Forest auch im Vorteil, da dieser unbeeinflusst von schiefen Datenverteilungen bleibt.
 - ein PowerTransformer wie er beispielsweise in Scikit learn enthalten ist, behebt das Problem mit der Schiefe und könnte zum Schluß mit dem Random Forest führen
 - Batch size variieren:
 - eine größere Batchsize führt zu ruhigerem Training, aber auch zu Overfitting. Aufpassen!

Erkenntnis:

- Eine Beschneidung der Features ist zunächst nicht ratsam und führt zu Performanceminderung.
- Letzlich haben sind die neuen Featureideen als nicht nützlich herausgestellt.
- Latex in VSCode zu implementieren ist nicht ganz einfach

Entscheidungen:

- Implimentation des Powertransformer zunächst überspringen

Schwierigkeiten / offene Fragen:

- Kann durch Featureoptimierung das Neuronale Netz mit meinen Parametermöglichkeiten über $R^2 = 0.8$ steigen?
- Schwierigkeiten Latex auf VS-Code zum laufen zu bringen
- ständig neues Auftauchen der Latexhilfsdateien in gitchanges trotz gitignore
- noch unklar, ob Zeitaufwand für Powertransformer lohnenswert

Eintrag 8 *circa: 8h*

Datum: 24.02.2026 **Titel:** Dokumentation **Aus Gruppentreffen:** [5]

Arbeitsschritte:

- bemerkt, dass ich durch gitignore einige Daten aus dem Git geworfen habe die noch benötigt werden
 - noch viel Verständnis nötig...
 - Überlegung ob betreffende Dateien neu pushen, oder Versuchen auf alte Gitversion zurück zu gehen
- Nach Überlegungen von gestern Versuche ich nun durch Variation der Batchsize das Ergebnis noch weiter zu pushen
 - Testen mit Batchsize = 128
 - Nach langer Rechenzeit: $R^2 = 0.7922$
 - Fazit: Wie bereits in 05_Neural_Network_Felix.ipynb festgestellt, hilft eine größere Batchsize nicht weiter.
 - Weiterhin mit Batchsize 32 arbeiten.
- Zusammenfassungen geschrieben:
 - 01_LASSO_Ridge
 - 02_Decision_Tree
 - 03_Ensemble
 - 04_kNN_Regression geschrieben
- Saubere Dokumentation des Notebooks
- Ausarbeitung des Modellaufrufs mit Ausgabe der erreichten Werte
- Analyse der Werte im Kontext lineare Regression (weit übertroffen), aber auch Random Forest Modellen (nicht ganz erreicht).
- Bugfix der [build_tuner_modell]-Funktion (dennoch muss vor Ausführung der nachfolgenden Zellen auf Initialisierung des Tuner gewartet werden)
-

Schwierigkeiten / offene Fragen:

- Fehlermeldung: "notebook controller is DISPOSED"
 - Versuch Kernel Restart, VS-code neustart -> Hilft vorerst
- versehentlich Dateien gelöscht, durch Hilfe von Flo aus Repo wieder gerettet

Eintrag 9 (ca. 7h)

Datum: 25.03.2026

Titel: Ergebnisse der Gruppe zusammentragen **Aus Gruppentreffen:** 5

Arbeitsschritte: Vergleich mit linearer Regression zu NN hinzugefügt. Wie in Gruppentreffen vereinbart:

- Lesen aller Notebooks
- Zusammenfassen der Neuroanalalen Netzwerk Notebooks
 - zunächst "Schmierzettel in Word"
 - jedes Neuronale Netz durchgegangen

- Ergebnisse und Gedanken zusammengetragen
- Begin schreiben der Ergebnis PDF
- weitere Latex Einstellungen
- Auseinandersetzung mit den Fähigkeiten der Tau-Class als mächtige Latexvorlage
- Zusammenfassung der nuller Notebooks von Björn leicht überarbeitet **Erkenntnis:**
- Für Latex ist Textstudios deutlich intuitiver als VS-Code.
-

Entscheidungen:

- einzelne Abschnitte zu den bearbeiteten Parametern unter Zusammenfassung der Erkenntnisse der Gruppenmitglieder anstatt einfaches herunterarbeiten der Notebooks

Schwierigkeiten / offene Fragen: Zunächst scheinbare Widersprüche in den Ergebnissen der Featureanalyse bezüglich der Sinnhaftigkeit des [lr_schueele].

- einzelne Betrachtung unter festhalten der restlichen Parameter ergab keine nennenswerte Verbesserung
- Es ist jedoch davon auszugehen, dass Simultanoptimierung der Hyperparameter durchaus profitiert
-

Eintrag 10 (ca. 9h)

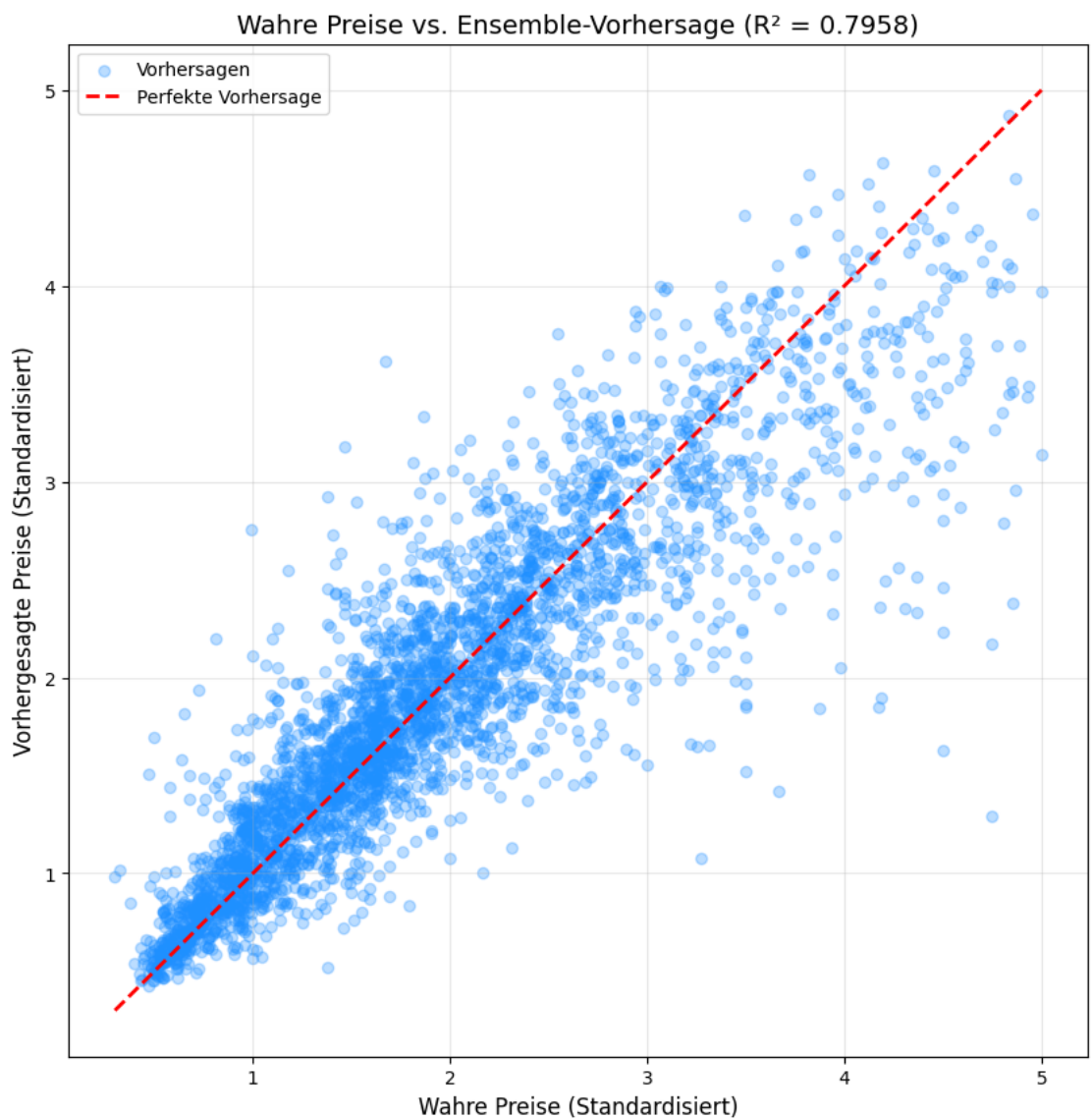
Datum: 26.03.2026

Titel: Gruppenergebnisse niederschreiben **Aus Gruppentreffen:** 5

Arbeitsschritte:

- Schreiben des Absatzes zur Explorativen Datenanalyse
 - zunächst Schwierigkeiten beim Einfügen einer Grafik (gelöst)
 - Aufteilung in Analyse, Bereinigung und Darstellung der Daten
- Lineare Regression
 - Baseline erklärt und Fehlerwerte und R^2 als Vergleich für spätere Modelle
 - LASSO und Ridge
- Decision Tree, Ensemble und K-Nearest neighbor als weiterer Vergleich und Messlatte für mögliche Performance erhalten jeweils eigenen Absatz
- Neuronale Netze
 - Vorgehen und Aufteilung erklärt
 - Ansätze zur Verbesserung des Neuronalen Netzes unter den Überthemen "Architektur" und "Featureengineering".
 - Architektur umfasst:
 - Layer
 - Regularisierung

- Aktivierungsfunktionen
- Hyperparameter
- Featureengineering beinhaltet die geschickte Wahl der richtigen für das training und Versuche der Neuschaffung von Parametern
- Absatz lineare Regression vs. Neuronale Netze - besonders viel Aufwand hinein stecken
- stellt sich heraus ich habe das stärkste Modell geschaffen
- (



)

- Vorbereitung auf Gruppentreffen
 - Fragen bezüglich der Zusammenfassung der Ergebnisse an die Gruppenmitglieder
- Gruppentreffen 6 (siehe Bericht GT &6)
 - persönliche Aufgaben:
 - fertig schreiben der Ergebnisse
 - hinzufügen der Personen die den jeweiligen Abschnitt bearbeitet haben
 - zusammen mit Björn Zusammenfassung der Gruppentreffen

- Formatierungen geklärt **Erkenntnis:**
 - Eine saubere Ergebnis PDF schreiben ist deutlich aufwendiger als erwartet

Schwierigkeiten / offene Fragen:

- git merge crash
 - Ständige Probleme durch ID Änderung durch ausführen des Codes anderer und pushen der ID-Änderung.
 - vorläufiger Fix kostet mich eine 1.5h, viele Nerven, aber hoffentlich keine Änderung an dem betroffenen Notebook
 - Tau class template bietet viele Möglichkeiten, hat aber auch sehr viele Schwierigkeiten mit denen es umzugehen gilt.->laufend neue Entdeckungen
 -
-

Eintrag 11 (ca. 9h)

Datum: 27.03.2026

Titel: Gruppenergebnisse niederschreiben **Aus Gruppentreffen:** 6

Arbeitsschritte:

- Hinzufügen der Namen der Gruppenmitglieder
- Einfügen von passenden Grafiken in Ergebnis Report (=ER)
- Kommentieren dieser Grafiken
- Rechtschreibkorrektur
- erneutes Korrekturlesen
- Referenzieren
- Zusammenfassung der Gruppentreffen mit Björn zsm abgabefertig gemacht
- Zusammenfassung dieses Logbuchs
 - generieren von Grafiken passend zu den Inhalten
 - ebenfalls mit Rechtschreibkorrektur
 - Bibliothek mit Referenzen aufbauen
 - Markdowndatei "Logbuch_Felix_Neumann" als Pdf exportiert und angehängt.
- erneutes Ausführen des Codes
- Titelseite vorbereitet und angehängt
- restliche Zeit mit Anhängen, zippen und hochladen **Erkenntnis:**
 - neues gelernt über Referenzierung in Latex
 - ich mache zu viele Rechtschreibfehler

Entscheidungen:

- 10 Finger schreiben lernen
- Markdownn in Latex anhängen

Schwierigkeiten / offene Fragen:

- ordentliches Verpacken aller Bestandteile
 - Rechtschreibkorrektur
-

6. QUELLEN UND HILFSMITTEL

LITERATUR

- [1] I.-K. Yeo und R. A. Johnson, „A new family of power transformations to improve normality or symmetry“, *Biometrika*, Jg. 87, Nr. 4, S. 954–959, 2000.
- [2] M. Kuhn und K. Johnson, *Applied Predictive Modeling*. Springer, 2013, Kapitel 3: Data Pre-processing.
- [3] L. Invernizzi, J. Long, F. Chollet, T. O’Malley und H. Jin, *Getting started with KerasTuner*, Zugriff: 21.03.2026, 2019. Adresse: https://keras.io/keras_tuner/getting_started/
- [4] S. J. Russell und P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th. Prentice Hall, 2020.
- [5] F. Rathgeber, *nbstripout – Strip output from Jupyter and IPython notebooks*, Zugriff: 11.02.2026, 2024. Adresse: <https://github.com/kynan/nbstripout>
- [6] Keras Team, *The Sequential model*, Zugriff: 22.03.2026, 2026. Adresse: https://keras.io/guides/sequential_model/